
Oscar A. Smalls, Jr.

COIN 496 Senior Portfolio

November 20, 2007

Table of Contents

1. Program Introductory Page
2. CD/DVD organizer sheet
3. Hard Copy of code with screen shots
4. Ethics Paper Introductory Page
5. Three Ethics Papers
6. Presentation Introductory Page
7. Advanced Networking Presentation
8. Operating System Presentation

Program Introductory Page

COIN 315 – Data Structures

- Program Name: triangle.cpp
- Grade Received: A
- Written in: Microsoft Visual C++ 2003
- Run program: triangle.exe

COIN 325 – Java I

- Program Name: TakeHomeTest2B.java
- Grade Received: A
- Written in: Java 1.4.2
- Run program: java TakeHomeTest2B

COIN 332 – Applied Networking

- Program Name: Blackjack.html
- Grade Received: A
- Written in: JavaScript
- Run program: Blackjack.html

COIN 333 – Applied Systems

- Program Name: problem-6.pl
- Grade Received: A
- Written in: Perl
- Run program: perl problem-6.pl

```

/*
  Student Name: Oscar A. Smalls, Jr.
  Program Name: triangle.cpp
  Creation Date: 2-4-2004
  Last Modified Date: 11-20-2007
  COIN Course: COIN 315
  Grade Received: A
  Comments regarding design: This program was written and compiled in Visual C++ 2003
*/

/*
  Program:      triangle.cpp
  Programmer:   Oscar A. Smalls, Jr.
  Date:         Feb. 4, 2004
  Desc:        This program solves the TEE puzzle for an equilateral triangle with 3 to
               6 tees on one side.  Puzzle is stored in array as a right triangle.
               Note that north west moves and south east moves are illegal in this
               right triangle matrix.

               Program solves the TEE puzzle by recursively 'finding the best move'
               where the last move will only leave one tee on the board.  When the last
               move leaves only one tee on the board, the recursive function unrolls
               itself and stores each move made in a Solution Board object.  The
               individual steps towards the solution are then displayed the user.

               The triangle matrix will be represented inside the computer as a square
               matrix.  There will be a border of one row for each edge of the square.
               For example, a 4 x 4 x 4 triangle will be represented as a square with
               width 6.

               +
              * *
             * * *
            * * * *

            becomes

            - - - - -
            -+***-
            -***-
            -**--
            -*---
            - - - - -

            where the '+' represents an empty space on the board, the '*' represents
            a tee, and the '-' represents invalid positions on the board.  However,
            the board must be displayed to the user as a triangle, not a square.
*/

#include <iomanip>
#include <iostream>

using namespace std;

const int boardSize = 8;      // Maximum board size (subscript 0 not used)
                               // subscripts 1 thru 6 can be used for boards up to width 6
                               // subscript 7 is used when 6 tees on edge of board to
                               // indicate an invalid position to jump to

const char INVALID = ' ';
const char TEE = '*';
const char EMPTY = '-';
const int MAX_BOARDS = 100; //Maximum number of solution boards that this program can
                             handle

struct boardType //holds current tee board

```

```
{
    char board[boardSize][boardSize];
    int maxTeesOnSide;
};

class solutionClass //holds solutions to tee board
{
public:
    void incrementCounter();
    bool addBoard(boardType bd);
    bool solved();
    int getCounter();
    solutionClass();
    boardType getBoard(int subscript);
    bool maxBoardsError();
private:
    boardType board[MAX_BOARDS];
    int counter; // counts number of boards to produce solution
    bool maxBoardsErrorFlag;
};

bool solutionClass::maxBoardsError()
{
    return maxBoardsErrorFlag;
};

boardType solutionClass::getBoard(int subscript)
{
    return board[subscript];
};

int solutionClass::getCounter()
{
    return counter;
};

bool solutionClass::solved()
{
    if (counter > 0)
        return true;
    else
        return false;
};

bool solutionClass::addBoard(boardType bd)
{
    if (counter < MAX_BOARDS)
    {
        board[counter++] = bd;
        return true;
    }
    else
    {
        maxBoardsErrorFlag = true;
        return false;
    }
};

solutionClass::solutionClass()
{
    counter = 0;
    maxBoardsErrorFlag = false;
};

void solutionClass::incrementCounter()
{

```

```
        counter++;
    };

void initializeBoard(boardType &bd);
void displaySystemBoard(boardType &bd);
void displayUserBoard(boardType &bd);
bool validMove(char value, boardType &bd, int row, int col);
void askForRowCol(int &row, int &col);
bool legalMoveExists(boardType &bd);
bool checkForLegalMove(boardType &bd, int row, int col);
bool makeMove(boardType &bd, int fromRow, int fromCol, int toRow, int toCol);
int findBestMove(boardType &bd, solutionClass &solutionBd);
int countTees(boardType &bd);
void askForMaxTeesOnSide(boardType &bd);
void showSolution(boardType initialBd, solutionClass solutionBd);

void main()
{
    boardType bd, initialBd;
    solutionClass solutionBd;
    int row, col;

    // Initialize the board
    askForMaxTeesOnSide(bd);
    initializeBoard(bd);

    do
    {
        // Display the initial board
        displaySystemBoard(bd);

        // Ask user for which tee should be removed to start the game
        cout << "Which tee should be removed to start the game?\n";
        askForRowCol(row, col);

        if (!validMove(TEE, bd, row, col))
            cout << "\nINVALID MOVE.\n";

    } while (validMove(TEE, bd, row, col) != true);

    cin.clear();
    cin.ignore(200, '\n');

    // Remove user's tee from the board
    bd.board[row][col] = EMPTY;

    initialBd = bd;

    // Obtain Solution Board object from the recursive 'Find the best move' function
    findBestMove(bd, solutionBd);

    if (solutionBd.maxBoardsError()) //if solution is more than array can handle
    {
        cout << "\nERROR: This program cannot compute a solution "
              << "with more than " << MAX_BOARDS << " moves.\n";
        return;
    }

    if (!solutionBd.solved()) //If there is no solution (more than one tee remains on
    board)
    {
        cout << "\nNO SOLUTION.\n\n";
        return;
    }

    // Display individual steps towards a solution to the user
```

```

    showSolution(initialBd, solutionBd);
};

void showSolution(boardType initialBd, solutionClass solutionBd)
{
    int i, j;
    char ch[80]; //used to wait for the ENTER key to be pressed to continue

    cout << "\nIt will take " << solutionBd.getCounter() << " moves to solve the puzzle.\n";

    j = 0;
    cout << "\n Initial Board:\n\n";
    displayUserBoard(initialBd);
    cout << endl;
    for (i = solutionBd.getCounter()-1; i >= 0; i--)
    {
        cout << "Press ENTER to continue."<< endl;
        cin.getline(ch,80);
        cin.clear();
        cout << "\n MOVE #" << ++j << ":\n\n";
        displayUserBoard(solutionBd.getBoard(i));
        cout << endl;
    };
    cout << endl;
}

void askForMaxTeesOnSide(boardType &bd)
{
    bd.maxTeesOnSide = 0;
    while (bd.maxTeesOnSide < 3 || bd.maxTeesOnSide > 6)
    {
        cout << "\nEnter number of tees on side of triangle (3 through 6): ";
        cin >> bd.maxTeesOnSide;
        cin.clear();
        cin.ignore(200, '\n');
        if (bd.maxTeesOnSide < 3 || bd.maxTeesOnSide > 6)
        {
            cout << "\nThis program cannot process a board with "
                << bd.maxTeesOnSide << " tees.\n";
        }
    };
};

int countTees(boardType &bd)
{
    int count = 0;

    for (int row = 1; row <= bd.maxTeesOnSide; row++)
        for (int col = 1; col <= bd.maxTeesOnSide; col++)
            if (bd.board[row][col] == TEE)
                count++;
    return count;
};

/*
FIND THE BEST MOVE FUNCTION

Argument to function is the board array
For each position in the array that contains a tee
    Check the positions that surround that tee
    If a position surrounding the tee contains a tee
        Check the next position beyond that tee
        If the position beyond the tee is empty
            Make a copy of the board
            Move the tee to the new position, removing the tee in the middle

```

```

        Check to see if there are any remaining moves left
        If there are no more moves
            Count the tees remaining
            If the count is equal to one, then add triangle to solution object
            Return the count to end iteration of the recursive function
        If there are moves remaining
            Call the 'Find the best move' function again,
            Passing the copy of the board
            If the 'Find the best move' function returned a value of one
                Add triangle to solution object
            Return number of tees remaining from the function
    */

int findBestMove(boardType &bd, solutionClass &solutionBd)
{
    boardType saveBoard;
    int max;

    for (int row=1; row <= bd.maxTeesOnSide; row++)
        for (int col=1; col <= bd.maxTeesOnSide; col++)
            if (bd.board[row][col] == TEE)
                for (int i = row-1; i <= row+1; i++)
                    for (int j = col-1; j <= col+1; j++)
                        if (!(row==i && col==j) && //cannot move to same position
                            !(row-i==1 && col-j==1) && //cannot move diagonally north ✓
                            west
                                !(i-row==1 && j-col==1) //cannot move diagonally south ✓
                                east
                                    )
                            if (bd.board[i][j] == TEE)
                                if (bd.board[i+(i-row)][j+(j-col)] == EMPTY)
                                    {
                                        saveBoard = bd;
                                        makeMove(saveBoard, row, col, i+(i-row), j+(j-col));
                                        if (!legalMoveExists(saveBoard))
                                            {
                                                max = countTees(saveBoard);
                                                if (max == 1)
                                                    {
                                                        solutionBd.addBoard(saveBoard);
                                                    }
                                                return max;
                                            }
                                        else
                                            {
                                                max = findBestMove(saveBoard, solutionBd);
                                                if (max == 1)
                                                    {
                                                        solutionBd.addBoard(saveBoard);
                                                        return max;
                                                    }
                                            }
                                    }
                                }

    return 0; // NO SOLUTION
};

bool makeMove(boardType &bd, int fromRow, int fromCol, int toRow, int toCol)
{
    if (bd.board[(fromRow+toRow)/2][(fromCol+toCol)/2] == TEE)
    {
        bd.board[(fromRow+toRow)/2][(fromCol+toCol)/2] = EMPTY;
        bd.board[fromRow][fromCol] = EMPTY;
        bd.board[toRow][toCol] = TEE;
        return true;
    }
}

```



```
    else
        return false;
};

bool checkForLegalMove(boardType &bd, int row, int col)
{
    for (int i = row-1; i <= row+1; i++)
        for (int j = col-1; j <= col+1; j++)
            if (!(i==row) && (j==col))
                if (bd.board[i][j] == TEE)
                    if (bd.board[i+(i-row)][j+(j-col)] == EMPTY)
                        return true;
    return false;
}

bool legalMoveExists(boardType &bd)
{
    bool legalMove = false;

    for (int row = 1; row <= bd.maxTeesOnSide; row++)
        for (int col = 1; col <= bd.maxTeesOnSide; col++)
        {
            if (bd.board[row][col] == TEE)
                legalMove = checkForLegalMove(bd, row, col);
            if (legalMove)
                return true;
        }
    return false;
};

void askForRowCol(int &row, int &col)
{
    cout << "Enter row: "; cout.flush();
    cin >> row;
    cout << "Enter col: "; cout.flush();
    cin >> col;
    if (!cin)
    {
        cin.ignore(100);
        cin.clear();
    }
};

bool validMove(char value, boardType &bd, int row, int col)
{
    if (row < 1 || col < 1 || row > bd.maxTeesOnSide || col > bd.maxTeesOnSide)
        return false;
    if (bd.board[row][col] != value)
        return false;
    return true;
};

void initializeBoard(boardType &bd)
{
    int row, col;

    for (col = 0; col < boardSize; col++)
        bd.board[0][col] = INVALID;
    for (row = 0; row < boardSize; row++)
        bd.board[row][0] = INVALID;
    for (row = 1; row <= bd.maxTeesOnSide; row++)
    {
        for (col = 1; row+col <= bd.maxTeesOnSide+1; col++)
            bd.board[row][col] = TEE;
        for (; col < boardSize; col++)
            bd.board[row][col] = INVALID;
    }
}
```

```

    };
    for (col = 0; col < boardSize; col++)
        bd.board[boardSize-1][col] = INVALID;
    for (row=bd.maxTeesOnSide+1; row < boardSize; row++)
        for (col=0; col < boardSize; col++)
            bd.board[row][col] = INVALID;
};

void displaySystemBoard(boardType &bd)
{
    cout << endl;
    cout << " ";
    for (int row = 0; row < boardSize; row++)
    {
        if (row > 0 && row <= bd.maxTeesOnSide)
            cout << " " << row;
        for (int col = 0; col < boardSize; col++)
        {
            if (row == 0 && col > 0 && col <= bd.maxTeesOnSide)
                cout << col << " ";
            else
                cout << bd.board[row][col] << " ";
        }
        cout << endl;
    };
};

void displayUserBoard(boardType &bd)
{
    for (int row = 1; row <= bd.maxTeesOnSide; row++)
    {
        for (int i = bd.maxTeesOnSide; i > row; i--)
            cout << " ";
        for (int counter = row; counter > 0; counter--)
            cout << bd.board[counter][row+1-counter] << " ";
        cout << endl;
    };
};

```

//OUTPUT

/*

Enter number of tees on side of triangle (3 through 6): 5

```

    1 2 3 4 5
1  * * * * *
2  * * * *
3  * * *
4  * *
5  *

```

Which tee should be removed to start the game?

Enter row: 1

Enter col: 1

It will take 13 moves to solve the puzzle.

Initial Board:

```

    * *
  * * *
* * * *
* * * * *

```

Press ENTER to continue.

MOVE #1:

```

      *
     *
    * *
   * * *
  * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #2:

```

      *
     *
    * *
   * * *
  * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #3:

```

      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #4:

```

      *
     *
    * *
   * * *
  * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #5:

```

      *
     *
    * *
   * * *
  * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #6:

```

      *
     *
    * *
   * * *
  * * * *
 * * * * *

```

Press ENTER to continue.

MOVE #13:

```

  _
 _ _
_ _ _
_ _ _
_ _ _

```

Press any key to continue
*/

```
1  /*
2  Student Name: Oscar A. Smalls, Jr.
3  Program Name: TakeHomeTest2B.java
4  Creation Date: 4-25-2005
5  Last Modified Date: 11-20-2007
6  COIN Course: COIN 325
7  Grade Received: A
8  Comments regarding design: This is a Java Application
9  */
10
11 // TakeHomeTest2B.java
12 // Oscar A. Smalls, Jr.
13 // April 25, 2005
14 // Exercise 11.24: This program reads in a date in the mm/dd/yyyy format
15 // and prints the date in the Month Day, Year format
16
17 import java.util.*;
18 import java.awt.*;
19 import java.awt.event.*;
20 import javax.swing.*;
21
22 public class TakeHomeTest2B extends JFrame {
23     private JLabel promptLabel;
24     private JTextField inputField;
25     private JTextArea outputArea;
26     private int month;
27     private String monthString;
28
29     // set up GUI and event handling
30     public TakeHomeTest2B()
31     {
32
33         super( "TakeHomeTest2B" );
34
35         Container container = getContentPane();
36         container.setLayout( new FlowLayout() );
37
38         promptLabel = new JLabel( "Enter date (mm/dd/yyyy) and press Enter"
39 );
40         container.add( promptLabel );
41
42         inputField = new JTextField( 20 );
43         inputField.addActionListener(
44             new ActionListener() { // anonymous inner class
45
46                 // handle text field event
47                 public void actionPerformed((ActionEvent event) )
48                 {
49
50                     StringTokenizer tokens =
51                         new StringTokenizer( event.getActionCommand(), "/" );
52
53                     if (tokens.countTokens() == 3)
54                     {
55                         month = Integer.parseInt(tokens.nextToken());
56
57                         switch(month)
58                         {
59                             case 1:
60                                 monthString = "January";
61                                 break;
62                             case 2:
63                                 monthString = "February";
64                                 break;
```

```
65         case 3:
66             monthString = "March";
67             break;
68         case 4:
69             monthString = "April";
70             break;
71         case 5:
72             monthString = "May";
73             break;
74         case 6:
75             monthString = "June";
76             break;
77         case 7:
78             monthString = "July";
79             break;
80         case 8:
81             monthString = "August";
82             break;
83         case 9:
84             monthString = "September";
85             break;
86         case 10:
87             monthString = "October";
88             break;
89         case 11:
90             monthString = "November";
91             break;
92         case 12:
93             monthString = "December";
94             break;
95         default:
96             break;
97     }
98
99     outputArea.setText(monthString + " " +
100         tokens.nextToken() + ", " +
101         tokens.nextToken());
102     }
103     else
104     {
105         outputArea.setText("Date format must be mm/dd/yyyy.")
106     }
107
108     }
109
110     } // end anonymous inner class
111
112     ); // end call to addActionListener
113
114     container.add( inputField );
115
116     outputArea = new JTextArea( 5, 20 );
117     outputArea.setEditable( false );
118     container.add( new JScrollPane( outputArea ) );
119     setSize( 275, 240 ); // set the window size
120     setVisible( true ); // show the window
121 }
122
123 // execute application
124 public static void main( String args[] )
125 {
126     TakeHomeTest2B application = new TakeHomeTest2B();
127     application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
128 }
129
```

```
130 } // end class TakeHomeTest2B  
131
```


TakeHomeTest2B

Enter date (mm/dd/yyyy) and press Enter

11/20/2007

November 20, 2007

```
<?xml version = "1.0"?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">

<!-- Student Name:          Oscar A. Smalls, Jr.      -->
<!-- Program Name:          Blackjack.html            -->
<!-- Creation Date:         12-12-2005                -->
<!-- Last Modified Date:    11-20-2007                -->
<!-- COIN Course:           COIN 332                  -->
<!-- Grade Received:        A                          -->
<!-- Comments regarding design:                        -->

<!-- Blackjack              -->
<!-- Oscar A. Smalls, Jr.   -->

<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <title>Blackjack</title>

    <style type="text/css">
      body {color: white}
    </style>

    <script type="text/javascript">
      <!--
      var currentCard = 0; // set currentCard to first card
      var compTotal; // stores computer's total value of cards
      var userTotal; // stores user's total value of cards

      // create the deck of cards
      var compCard = new Array(6);
      var userCard = new Array(6);
      var FACE = 0;
      var SUIT = 1;
      var faces = ["A","2","3","4","5","6","7","8","9","T","J","Q","K"];
      var suits = ["h","d","c","s"];
      var deck = new Array(52);
      for (var count = 0; count < deck.length; count++)
      {
        deck[count] = new Array(2);
        deck[count][FACE] = faces[count%13];
        deck[count][SUIT] = suits[Math.floor(count/13)];
      }
      var faceDown = new Array(2);
      faceDown[FACE] = "";
      faceDown[SUIT] = "";

      // wait for user to press New Game button

      function clean()
      {
        usercard1.innerHTML="<img src=\"blank.gif\" />";
        usercard2.innerHTML="<img src=\"blank.gif\" />";
        usercard3.innerHTML="<img src=\"blank.gif\" />";
        usercard4.innerHTML="<img src=\"blank.gif\" />";
        usercard5.innerHTML="<img src=\"blank.gif\" />";
        compcard1.innerHTML="<img src=\"blank.gif\" />";
        compcard2.innerHTML="<img src=\"blank.gif\" />";
        compcard3.innerHTML="<img src=\"blank.gif\" />";
        compcard4.innerHTML="<img src=\"blank.gif\" />";
        compcard5.innerHTML="<img src=\"blank.gif\" />";
      }

      function shuffle()
      {
        currentCard = 0;
```

```
        for (var first = 0; first < deck.length; first++)
        {
            var second = Math.floor( Math.random() * 52);
            var faceHold = deck[first][FACE];
            var suitHold = deck[first][SUIT];
            deck[first][FACE] = deck[second][FACE];
            deck[first][SUIT] = deck[second][SUIT];
            deck[second][FACE] = faceHold;
            deck[second][SUIT] = suitHold;
        }
    }

function dealCard()
{
    var card = new Array(2);
    card[FACE] = "";
    card[SUIT] = "";
    if (currentCard < deck.length)
    {
        card[FACE] = deck[currentCard][FACE];
        card[SUIT] = deck[currentCard][SUIT];
        currentCard++;
    }
    return card;
}

function newGame()
{
    // reset the user interface
    blackjack.hitMeButton.disabled = "";
    blackjack.standButton.disabled = "";

    // shuffle the cards
    shuffle();

    // display computer's first card
    compSetCard(1, dealCard());

    // display rest of computer's cards face down
    compSetCard(2, faceDown);
    compSetCard(3, faceDown);
    compSetCard(4, faceDown);
    compSetCard(5, faceDown);

    // display user's first two cards
    userSetCard(1, dealCard());
    userSetCard(2, dealCard());

    // display rest of user's cards face down
    userSetCard(3, faceDown);
    userSetCard(4, faceDown);
    userSetCard(5, faceDown);

    gameOver(); // check to see if game is over
}

function getCardValue(face)
{
    switch(face)
    {
        case "A":    return 1;
        case "2":    return 2;
        case "3":    return 3;
        case "4":    return 4;
        case "5":    return 5;
        case "6":    return 6;
```

```

        case "7":    return 7;
        case "8":    return 8;
        case "9":    return 9;
        case "T":    return 10;
        case "J":    return 10;
        case "Q":    return 10;
        case "K":    return 10;
        default :    return 0;
    }
}

function compSetCard(i, card)
{
    compCard[i] = getCardValue(card[FACE]);
    filename = getFilename(card);
    switch(i)
    {
        case 1: compcard1.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 2: compcard2.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 3: compcard3.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 4: compcard4.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 5: compcard5.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
    }
}

function getFilename(card)
{
    if (card[FACE] + card[SUIT] != "")
        return card[FACE] + card[SUIT] + ".gif";
    else
        return "back.gif";
}

function userSetCard(i, card)
{
    userCard[i] = getCardValue(card[FACE]);
    filename = getFilename(card);
    switch(i)
    {
        case 1: usercard1.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 2: usercard2.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 3: usercard3.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 4: usercard4.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
        case 5: usercard5.innerHTML="<img src=\""+filename+"\" width=\"71\" height=\"96\"/>";
                break;
    }
}

```

```
function hasAce(cardHand)
{
    if (cardHand[1] == 1 || cardHand[2] == 1 || cardHand[3] == 1 || cardHand
[4] == 1 || cardHand[5] == 1)
        return true;
    else
        return false;
}

function calcTotal()
{
    compTotal = compCard[1] + compCard[2] + compCard[3] + compCard[4] +
compCard[5];
    userTotal = userCard[1] + userCard[2] + userCard[3] + userCard[4] +
userCard[5];
    compGrandTotal.innerHTML = compTotal+"";
    userGrandTotal.innerHTML = userTotal+"";
    if (hasAce(compCard) && (compTotal+10 < 21))
        compGrandTotal.innerHTML = compTotal+" or "+(compTotal+10);
    if (hasAce(userCard) && (userTotal+10 < 21))
        userGrandTotal.innerHTML = userTotal+" or "+(userTotal+10);
    if (hasAce(compCard) && (compTotal+10 == 21))
        compGrandTotal.innerHTML = (compTotal+10)+"";
    if (hasAce(userCard) && (userTotal+10 == 21))
        userGrandTotal.innerHTML = (userTotal+10)+"";
}

function gameOver()
{
    // calculate computer total and user total
    calcTotal();

    // does user have blackjack?
    if (    ((userCard[1] == 1) && (userCard[2] == 10))    ||
        ((userCard[1] == 10) && (userCard[2] == 1))    )
    {
        userWins("BLACKJACK! "); // user wins
        return true;
    }

    // does computer have blackjack?
    if (    ((compCard[1] == 1) && (compCard[2] == 10))    ||
        ((compCard[1] == 10) && (compCard[2] == 1))    )
    {
        compWins("BLACKJACK! "); // computer wins
        return true;
    }

    // is user's total greater than 21?
    if (userTotal > 21)
    {
        compWins("BUST! "); // computer wins
        return true;
    }

    // does user have 5 card trick?
    if ((userCard[5] > 0) && (userTotal <= 21))
    {
        userWins("5 CARD TRICK! "); // user wins
        return true;
    }

    // is computer's total greater than 21?
    if (compTotal > 21)
    {

```

```
        userWins("BUST! "); // user wins
        return true;
    }

    // does computer have 5 card trick?
    if ((compCard[5] > 0) && (compTotal <= 21))
    {
        compWins("5 CARD TRICK! "); // computer wins
        return true;
    }

    // is user's total equal to 21?
    if (userTotal == 21)
    {
        userWins(""); // user wins
        return true;
    }

    // is computer's total equal to 21?
    if (compTotal == 21)
    {
        compWins(""); // computer wins
        return true;
    }

    // are all computer cards displayed?
    if (compCard[5] > 0)

        // is computer's total greater than user's total?
        if (compTotal > userTotal)
        {
            compWins(""); // computer wins
            return true;
        }
        else
        {
            userWins(""); // user wins
            return true;
        }

    return false; // otherwise, game is not over
}

function userWins(message)
{
    blackjack.hitMeButton.disabled = "disabled";
    blackjack.standButton.disabled = "disabled";
    window.alert(message+"Player wins.");
}

function compWins(message)
{
    blackjack.hitMeButton.disabled = "disabled";
    blackjack.standButton.disabled = "disabled";
    window.alert(message+"House wins.");
}

function hitMe()
{
    // is user's 3rd card displayed?
    if (userCard[3] == 0)
    {
        userSetCard(3, dealCard()); // display user's 3rd card
        gameOver(); // check to see if game is over
    }
    else
}
```

```

        {
            // is user's 4th card displayed?
            if (userCard[4] == 0)
            {
                userSetCard(4, dealCard()); // display user's 4th card
                gameOver(); // check to see if game is over
            }
            else
            {
                // is user's 5th card displayed?
                if (userCard[5] == 0)
                {
                    userSetCard(5, dealCard()); // display user's 5th card
                    gameOver(); // check to see if game is over
                }
            }
        }
    }

function stand()
{
    // does user have an ace card?
    if (hasAce(userCard))
    {
        // add 10 to the user's total if it
        // does not take the total over 21
        userTotal = userTotal + 10;
        if (userTotal > 21)
            userTotal = userTotal - 10;
        userGrandTotal.innerHTML = userTotal+"";
    }

    // display computer's 2nd card
    compSetCard(2, dealCard());

    if (!gameOver()) // if game is not over
    {
        // display computer's 3rd card
        compSetCard(3, dealCard());

        if (!gameOver()) // if game is not over
        {
            // display computer's 4th card
            compSetCard(4, dealCard());

            if (!gameOver()) // if game is not over
            {
                // display computer's 5th card
                compSetCard(5, dealCard());

                gameOver(); // check to see if game is over
            }
        }
    }
}

// ->
</script>
</head>
<body style="background-color: #006400;" onload="clean()">
    <form action="" id="blackjack" name="blackjack">
        <h1 align="center">Blackjack</h1>
        <table cellpadding="5" align="center">
            <thead>
                <tr>
                    <th></th>

```

```

        <th></th>
        <th></th>
        <th></th>
        <th></th>
        <th></th>
        <th>Points</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td align="right"><strong>Computer</strong></td>
        <td><p id="compcard1"></p></td>
        <td><p id="compcard2"></p></td>
        <td><p id="compcard3"></p></td>
        <td><p id="compcard4"></p></td>
        <td><p id="compcard5"></p></td>
        <td align="center"><strong><p id="compGrandTotal"></p></strong></td>
    </tr>
    <tr>
        <td align="right"><strong>User</strong></td>
        <td><p id="usercard1"></p></td>
        <td><p id="usercard2"></p></td>
        <td><p id="usercard3"></p></td>
        <td><p id="usercard4"></p></td>
        <td><p id="usercard5"></p></td>
        <td align="center"><strong><p id="userGrandTotal"></p></strong></td>
    </tr>
</tbody>
</table>
<table cellpadding="10" align="center">
    <tr>
        <td><input type="button" name="hitMeButton" value="Hit me" disabled=
"disabled" onclick="hitMe()" /></td>
        <td><input type="button" name="standButton" value="Stand" disabled=
"disabled" onclick="stand()" /></td>
    </tr>
    <tr>
        <td colspan="2" align="center"><input type="button" value="New Game"
onClick="newGame()"></td>
    </tr>
</table>
</form>
</body>
</html>

```




problem-6.pl

```
#!/usr/bin/perl
# problem-6.pl
use warnings;
use strict;

# Student Name: Oscar A. Smalls, Jr.
# Program Name: problem-6.pl
# Creation Date: 4-14-2007
# Last Modified Date: 11-20-2007
# COIN Course: COIN 333
# Grade Received: A
# Comments regarding design:

# define variables used in program

my (@grade_book, @project2, $pattern_PROJECT2);
my ($student, $grade, $element, %student_grades, %missing_students);
my ($pretext, $posttext, @temp_array);

# array holding student grades with the first line being column labels
@grade_book= (
    "User Name, First, Last Name, HW1, HW2, HW3, PROJECT2,P1",
    "gunderwood,Gary,Underwood,90,80,70,,92,",
    "fworthy,Fred,Worthy,80,90,70,,81,",
    "jbriner,Jack,Briner,95,85,75,,92,",
    "ylin,Joseph,Lin,70,80,90,,99,",
    "jroberts,James,Roberts,75,85,95,,78,",
);

# array with grades for students who submitted PROJECT2
@project2=(
    "jbriner,0",
    "fworthy,82",
    "gunderwood,100",
    "jroberts,80",
    "sperrine,30",
);

# pattern is used to match against lines in @grade_book
# $3 will contain the value in the column PROJECT2, including the comma
# $1 will contain all the text before the PROJECT2 column, and $4 all the text after

$pattern_PROJECT2='^((.*?,){6})(.*?,)(.*)';

print "\n";
print 'Beginning @grade_book:';
print "\n@grade_book\n";

# put student and their grades from project2 in a hash
foreach $element (@project2)
{
    ($student, $grade) = $element =~ /(\w+),(\w+)/;
    $student_grades{$student} = $grade;
}
```

```

}

foreach $element (@grade_book)
{
    if ($element =~ '^[a-z]+') # if element matches student record and not
    field names
    {
        $student = $1;
        if (exists $student_grades{$student}) # if student has grade for
PROJECT2
        {
            # update student's PROJECT2 grade in gradebook
            $element =~
s/$pattern_PROJECT2/$1$student_grades{$student},$4/;

            # remove student from project2 hash
            delete $student_grades{$student};
        }
        else
        {
            $missing_students{$student}++; # note student missing
PROJECT2
        }
    }
}

print "\n";
print 'Ending @grade_book:';
print "\n@grade_book\n";

@temp_array = %missing_students; # get total number of students missing PROJECT2
if (@temp_array > 0)
{
    print "\nThe following students did not submit PROJECT2: ";
    foreach $student (keys %missing_students)
    {
        print "$student ";
    }
}

@temp_array = %student_grades; # get total number of students not in gradebook
if (@temp_array > 0)
{
    print "\nThe following students were not found in the grade book: ";
    foreach $student (keys %student_grades)
    {
        print "$student ";
    }
}

```

```
Command Prompt
C:\Transfer\Portfolio>perl problem-6.pl

Beginning @grade_book:
User Name, First, Last Name, HW1, HW2, HW3, PROJECT2, P1 gunderwood, Gary, Underwood, 90, 80, 70, 92, fuorthy, Fred, Worthy, 80, 90, 70, 81, jbriner, Jack, Briner, 95, 85, 75, 92, ylin, Joseph, Lin, 70, 80, 90, 99, jroberts, James, Roberts, 75, 85, 95, 78,

Ending @grade_book:
User Name, First, Last Name, HW1, HW2, HW3, PROJECT2, P1 gunderwood, Gary, Underwood, 90, 80, 70, 100, 92, fuorthy, Fred, Worthy, 80, 90, 70, 82, 81, jbriner, Jack, Briner, 95, 85, 75, 0, 92, ylin, Joseph, Lin, 70, 80, 90, 99, jroberts, James, Roberts, 75, 85, 95, 80, 78,

The following students did not submit PROJECT2: ylin
The following students were not found in the grade book: sperrine
C:\Transfer\Portfolio>
```

Ethics Paper Introductory Page

“Music Industry vs. Music Fans”

- COIN 332 – Applied Networking
- Grade Received: A

“The Ethics of Connecting to Open Wireless Access Points”

- COIN 326 – Java II
- Grade Received: A

“Video Game Violence”

- COIN 431 – Operating Systems
- Grade Received: A